

Proyecto Final de Ingeniería Mecatrónica

Alumnos:

- Salto, Alejo Daniel
- Vazquez, Nahir Alanis

Docentes:

- Blanca, Ezequiel
- Lukaszewicz, Cristian
- Szombach, Juan

Índice

RESUMEN	3
INTRODUCCIÓN.....	3
DISEÑO	4
PRESUPUESTO	9
CRONOGRAMA	11
FUNCIONAMIENTO	14
TESTEO.....	16
CONCLUSIONES	17
ANEXO 1: Hojas de datos de los componentes utilizados	18

RESUMEN

El presente informe explica en detalle el proceso de diseño, desarrollo y funcionamiento de un prototipo de comedero inteligente para mascotas, ComederoSmart, incluyendo el diseño de la estructura, los esquemas de conexiones, componentes y materiales utilizados, presupuesto, cronograma de tareas realizadas, el software elaborado y problemas enfrentados y sus respectivas soluciones, y posibles aspectos a profesionalizar en caso de buscar comercializarlo.

INTRODUCCIÓN

El proyecto ComederoSmart nace para resolver una problemática común que se da en los hogares con más de una mascota. A la hora de alimentarlos, sobre todo si tienen diferentes necesidades, ya sea por salud, peso, porque se trata de razas diferentes, o de diferentes edades, se encuentran diversas dificultades, volviéndose una tarea tediosa que se complica aún más si los animales son reactivos o conflictivos, o si se debe estar fuera de casa por varias horas. En muchas situaciones, además, es necesario impedir que una mascota coma el alimento que le corresponde a la otra, lo que suele obstaculizar la tarea aún más.

El proyecto consiste de un comedero inteligente para mascotas, que tiene la capacidad de reconocer a cada una de las mascotas del hogar gracias a la cámara integrada con visión artificial, brindándoles la cantidad necesaria del alimento que les corresponda según la información suministrada por el usuario desde la aplicación, y evitando que otra mascota, ya sea perteneciente a la casa o no, pueda consumirla. La aplicación para celular, además, permite almacenar toda la información de los animales para llevar un registro personalizado de las comidas de cada uno. También son configurables los horarios en los que debe brindársele la comida a cada animal, aunque el comedero se abre cuantas veces el animal se acerque, siempre que haya en él comida y el animal no haya acabado ya su ración del día. Además, podrá configurarse si es necesario que el animal haga ayuno, o si es requerido darle otro tipo de alimentos y el comedero no debe dispensarle su ración.

Para utilizar el comedero se le asigna automáticamente un plato a cada animal, y una vez hecho esto, sólo se le permitirá comer de ese plato al animal correspondiente, cerrándose al acercarse otro. Además, gracias a las celdas de carga bajo cada plato se puede racionalizar la comida de cada animal con la precisión adecuada. Por último, el comedero tiene un parlante sobre cada plato, que llama el nombre del animal cuyo plato corresponde, mediante un audio configurable por el dueño por la aplicación, y cuenta con sensores ultrasónicos para indicar el nivel restante de alimento en el dispensador.

DISEÑO

El comedero se diseñó con los laterales en diagonal, como se puede ver en la figura 1, para brindar más separación a los platos de cada mascota, en caso de que se acerquen a comer al mismo tiempo. El comedero tiene dos cajones que albergan los platos en los que se alimenta a cada mascota, como se aprecia en la figura 2. Estos cajones cuentan con dos motores de CC de 3 V a 13 V con sus respectivas ruedas para movilizarlos, controlados por el driver L298N, además de contar con guías en la estructura para que su movimiento no se desvíe, y sensores de final de carrera que controlan el comienzo y el final del movimiento. Además, dentro de cada cajón se colocó debajo de cada plato de comida la celda de carga correspondiente, junto al controlador HX711.

Para la estructura del comedero, si bien no se realizaron cálculos rigurosos pues teníamos libertad, se contempló que las diagonales fueran tal que los cajones entraran por completo al cerrarse, y no se chocaran con la pared del fondo.

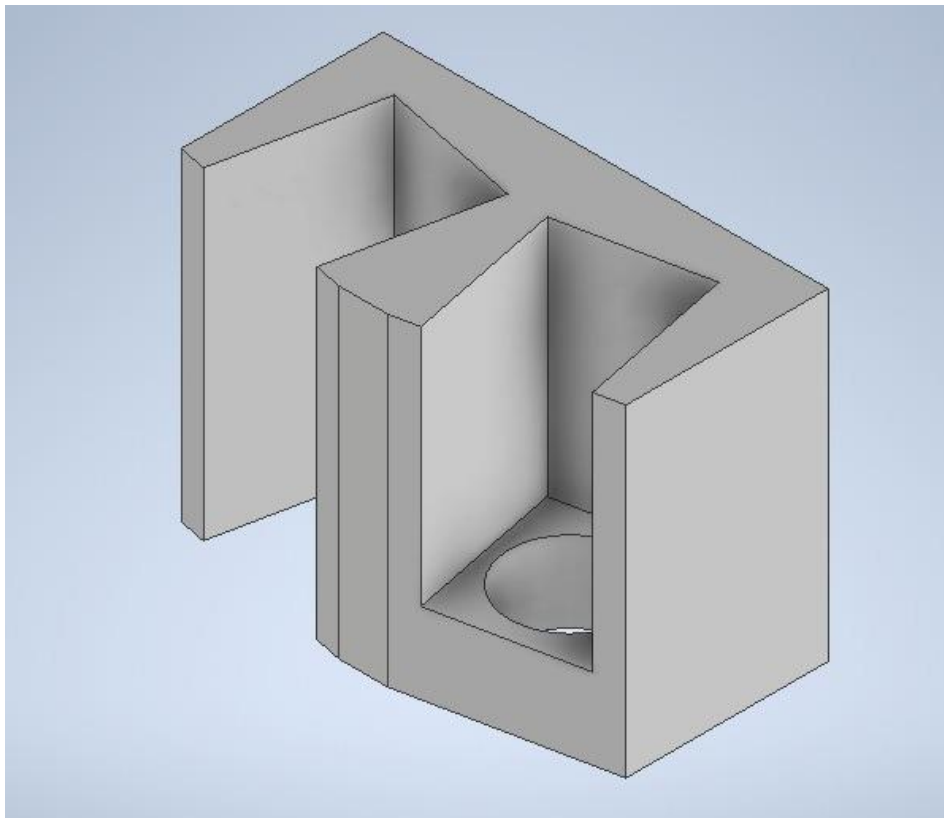


Figura 1. Diseño de la estructura con los cajones cerrados.

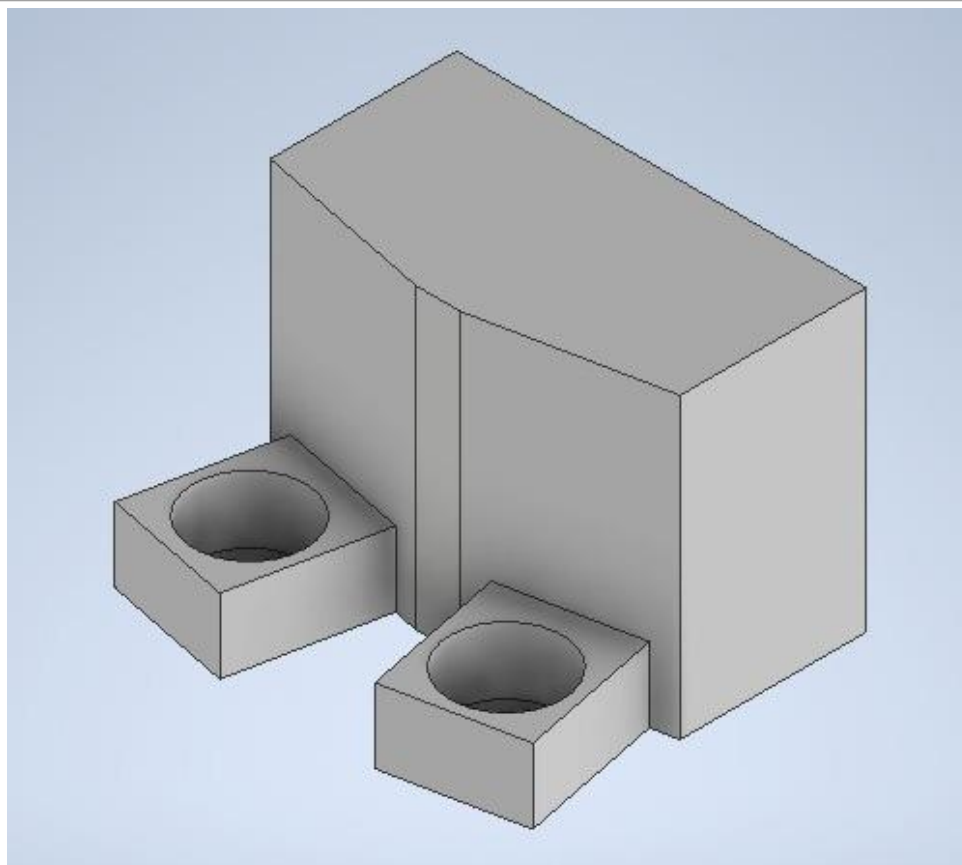


Figura 2. Estructura con los cajones abiertos.

Para dispensar la comida, se decidió utilizar el mecanismo de tornillo sin fin, que se diseñó junto a sus respectivos engranajes y se imprimió en 3D. Los diseños pueden apreciarse en las figuras 3 y 4. Los tornillos sin fin se fijaron en la estructura con un rodamiento para evitar que la comida los moviera de su eje, y se hicieron girar con un motor paso a paso de 5V y 512 pasos por vuelta, controlados por el driver ULN2003. Para diseñar los engranajes se partió del módulo y la relación de transmisión deseadas, que se eligieron por comodidad

$i = \frac{1}{3}$, $M = 1,5 \text{ mm}$. A partir de esto, se calcularon todos los parámetros necesarios, vistos en la tabla 1:

Tabla 1. Parámetros para el diseño de los engranajes.

Parámetro	Engranaje menor	Engranaje mayor
$Z = \text{número de dientes}$	20	60
$D_p = Z * M \text{ [mm]}$	30	90
$D_{ext} = D_p + 2 * M \text{ [mm]}$	33	93
$D_{int} = D_p - 2 * 1,66 * M$	26,5	86,5
$\alpha = 20^\circ, D_b = D_p * \cos(\alpha)$	28,19	84,57
$W = 1,166 M$	1,750	1,750
$S_k = 0,166 M$	0,250	0,250

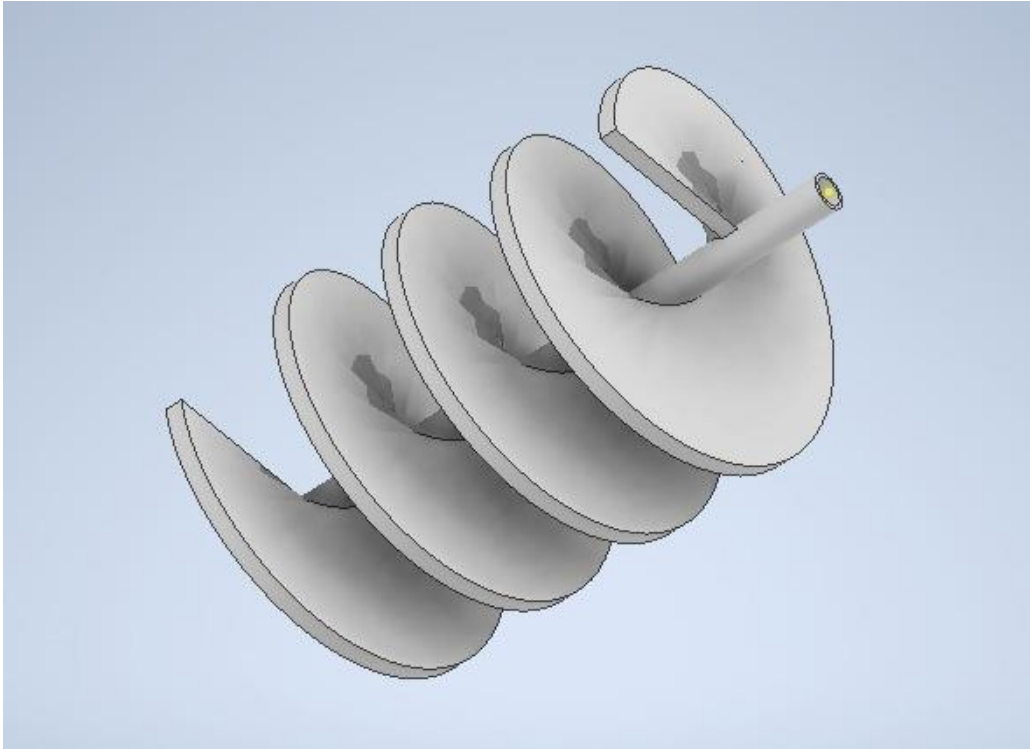


Figura 3. Diseño de los tornillos sin fin que se usaron para dispensar.

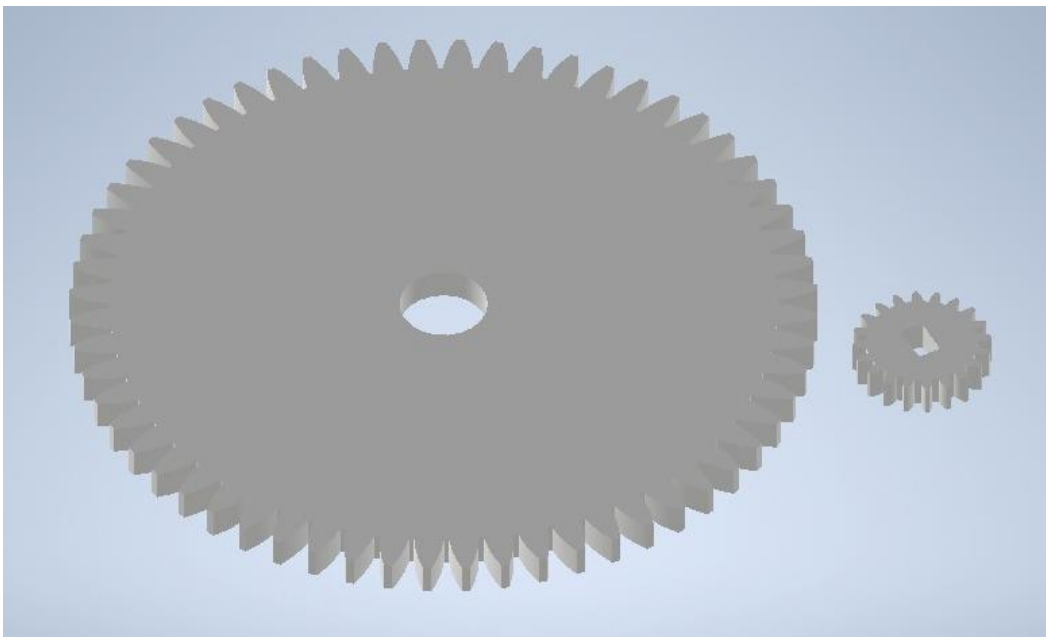


Figura 4. Diseño de los engranajes que mueven los tornillos sin fin.

Sobre cada cajón se colocó una ESP32 CAM mediante la cual se realizó la visión artificial. Además, en los pines que sobraban en cada cámara se conectó el sensor ultrasónico para medir el nivel de comida en cada dispensador, como se aprecia en la figura 5.

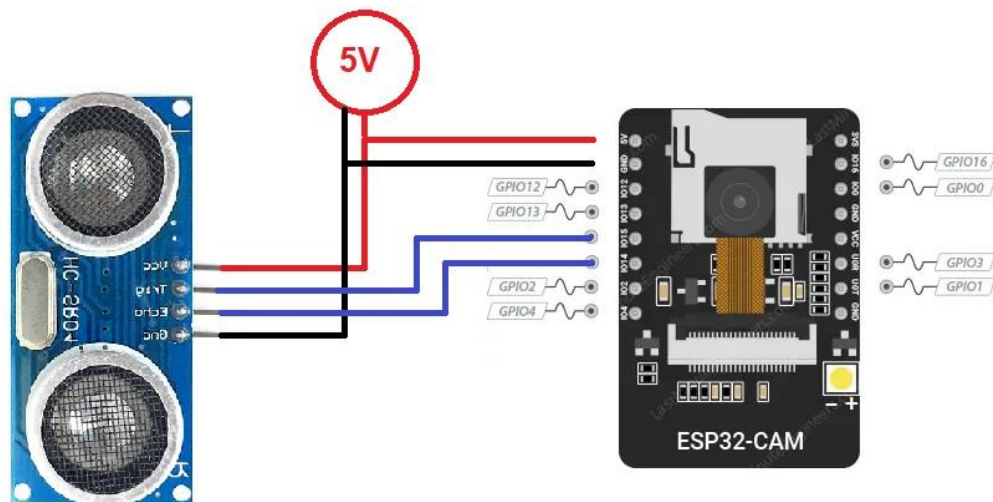


Figura 5. Esquema de conexión de las ESP32-CAM.

De las hojas de datos en el anexo 1, tenemos que:

Las celdas de carga se alimentan con 5 V y el amplificador HX711 consume aproximadamente 2 mA. Los motores de CC consumen hasta 1A con carga máxima, y e L298N aproximadamente 30 mA. El módulo ULN2003 consume aproximadamente 300 mA con el motor28BYJ-48 funcionando. Los finales de carrera son despreciables, y los microcontroladores consumen 200 mA la ESP32 y 300 mA la ESP32-CAM.

El consumo de corriente estimado es, entonces:

$$2 \times (2 \text{ mA} + 1000 \text{ mA} + 30 \text{ mA} + 300 \text{ mA} + 200 \text{ mA} + 300 \text{ mA}) = 3664 \text{ mA} = 3.664 \text{ A}$$

Finalmente, toda la electrónica fue controlada por una ESP32, cuyo PINOUT se muestra en la figura 6. El L298N fue alimentado con una fuente de 12V 2A, mientras que los módulos ULN2003 para los motores paso a paso fueron alimentados por una fuente de 5V 2A y los microcontroladores (las ESP32 CAM y la ESP32) fueron alimentados con otra fuente de 5V 2A.

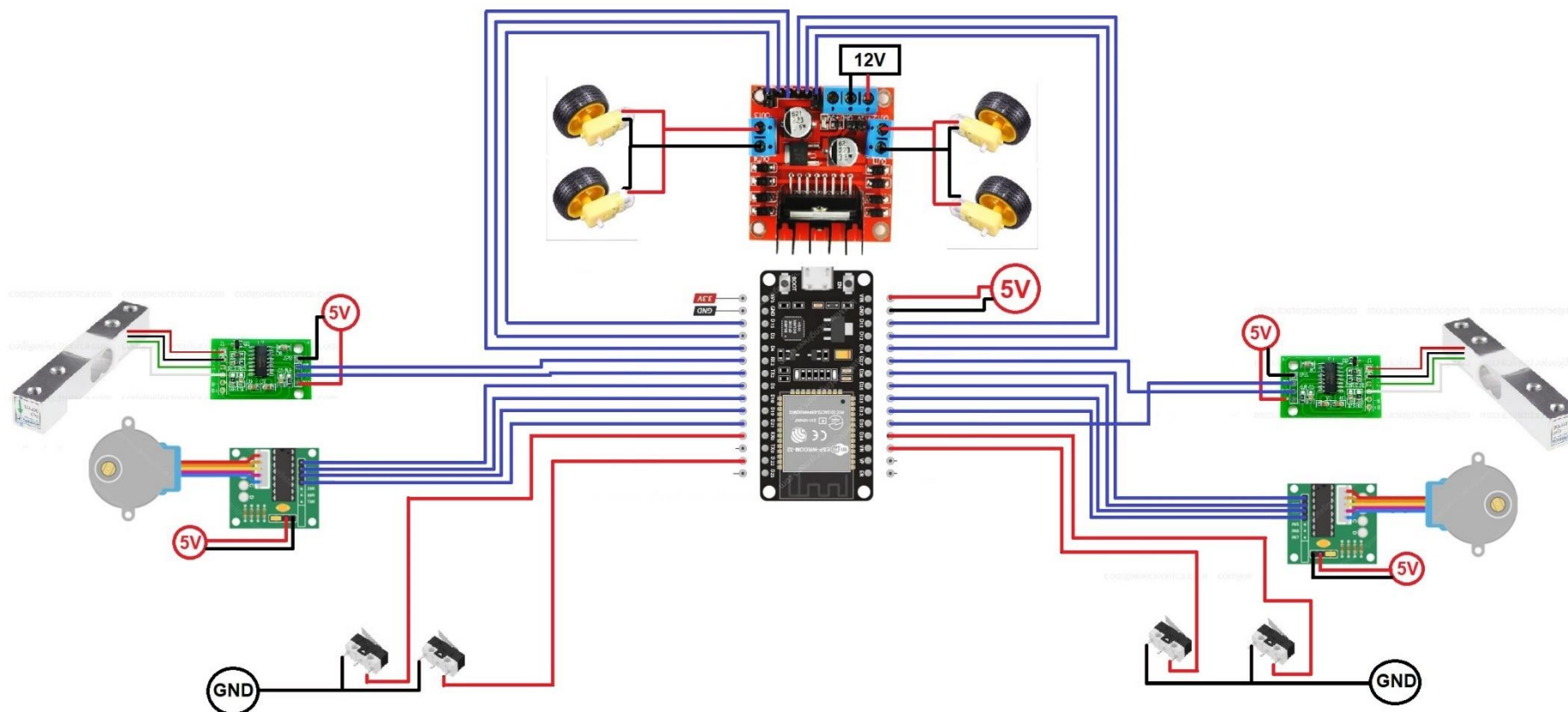


Figura 6. Esquema de conexiones de la ESP32.

PRESUPUESTO

El presupuesto estimado al inicio del proyecto, junto con los componentes, fue el mostrado en la tabla 2.

Tabla 2. Presupuesto y componentes estimados.

Componente	Proveedores	Precio (ARS)	Precio (USD)	Unidades	Costo total (ARS)	Costo total (USD)	Pines necesarios
Cámara	Starware	15000	11,54	2	30000	23,08	-
Sensor ultrasónico	Starware	700	0,54	2	1400	1,08	6
Celdas de carga	Starware	3500	2,69	2	7000	5,38	6
Paso a paso	RC Electrónica	6900	5,31	2	13800	10,62	10
Motor DC	EMakers	1700	1,31	4	6800	5,23	8
Final de carrera	RC Electrónica	750	0,58	4	3000	2,31	4
Buzzer + Transistor	EMakers	1200	0,92	2	2400	1,85	4
Led	EMakers	100	0,08	2	200	0,15	2
Esp32	Starware	7000	5,38	1	7000	5,38	-
Impresión 3D de tornillos sin fin	anter3d	5000	3,85	2	10000	7,69	-
Impresión 3D engranajes	anter3d	2000	1,54	4	8000	6,15	-
Impresión 3D guías para los cajones	anter3d	2000	1,54	2	4000	3,08	-
Material estructura	Artesanías MG	30000	23,08	1	30000	23,08	-
Total					123600	95,08	40

En la tabla 3, sin embargo, pueden apreciarse los componentes reales utilizados, y el presupuesto final, que fue prácticamente el doble que el planteado al inicio:

Tabla 3. Presupuesto y componentes reales.

Componente	Proveedores	Precio (ARS)	Precio (USD)	Unidades	Costo total (ARS)	Costo total (USD)	Pines necesarios
Cámara	Starware	15000	11,54	2	30000	23,08	-
Sensor ultrasónico	Starware	700	0,54	2	1400	1,08	6
Celdas de carga	Starware	3500/4500	2,69	2/3	20500	18.63	6
Paso a paso	RC Electrónica	6900	5,31	2	13800	10,62	10
Motor DC	EMakers	1700	1,31	4	6800	5,23	8
Final de carrera	RC Electrónica	750	0,58	4	3000	2,31	4
Led	EMakers	100	0,08	2	200	0,15	2
Esp32	Starware	7000	5,38	1	7000	5,38	-
Impresión 3D de tornillos sin fin	anter3d	5000	3,85	2	10000	7,69	-
Impresión 3D engranajes	anter3d	2000	1,54	4	8000	6,15	-
Material estructura	Artesanías MG	30000	23,08	1	30000	23,08	-
Bulonería	TodoBulón	-	-	-	30000	27.27	-
Estética	-	-	-	-	10000	9.09	-
Cables, fuentes, pineras, placas	FullElectrónica	-	-	-	30000	27.27	-
Total					200700	182.67	40

CRONOGRAMA

El cronograma tentativo propuesto al inicio del proyecto puede verse en las columnas con la etiqueta “Estimado/a” en la tabla 4, mientras que el cronograma real en el que se fueron completando las tareas puede verse en las columnas con la etiqueta “Real”. Para llegar con los tiempos de la cursada, a medida que fueron surgiendo imprevistos se fueron dejando tareas de lado, para realizar a la hora de la entrega final, como lo fueron todas las dedicadas a la creación de la aplicación. Además, algunas funciones específicas de la aplicación, como la carga de audios o imágenes, fueron modificadas en el prototipo entregado, ya que la dificultad que suponían no se justificaba por su poca correlación con los contenidos de la materia. Sin embargo, estas funciones y algunos detalles que se fueron notando a lo largo del proyecto, se proponen como mejoras para el producto final.

Tabla 4. Diagrama de Gantt del proyecto.

TAREA	RESPONSABLE/S	PROGRESO	INICIO ESTIMADO	FIN ESTIMADO	HORAS ESTIMADAS	DEPENDENCIAS	INICIO REAL	FIN REAL	HORAS REALES	PROBLEMAS
Diseño de la estructura										
Investigación sobre mecanismos y modelos utilizados	Salto y Vazquez	100%	25-8-24	28-8-24	8	-	25-8-24	26-8-24	8	Ninguno
Propuestas de diseños posibles	Salto y Vazquez	100%	28-8-24	30-8-24	8	Investigación sobre mecanismos y modelos utilizados	28-8-24	30-8-24	8	Ninguno
Decisión final y dibujo CAD 3D	Vazquez	100%	30-8-24	1-9-24	5	Propuestas de diseños posibles	30-8-24	1-9-24	5	Ninguno
Selección y compra de materiales	Salto	25%	1-9-24	3-9-24	3	Decisión final y dibujo CAD 3D	1-9-24	5-12-24	15	Viajes extra a la bulonera y a la maderera
Construcción de la estructura	Salto y Vazquez	0%	3-9-24	15-9-24	40	Selección y compra de materiales	3-9-24	13-12-24	60	Terminación del lado derecho
Desarrollo mecánico										
Selección de componentes necesarios	Salto y Vazquez	100%	1-9-24	3-9-24	3	Decisión final y dibujo CAD 3D	01-09-24	03-09-24	3	Ninguno
Diseño y desarrollo mecanismo de dispenser	Salto y Vazquez	0%	3-9-24	23-9-24	20	Construcción de la estructura	11-09-24	11-12-24	30	Demora en la impresión 3d, construcción del lado derecho, porosidad de la comida, desviación del eje

Desarrollo mecanismo de cajones	Salto y Vazquez	0%	23-9-24	3-10-24	20	Construcción de la estructura	23-09-24	08-12-24	40	Problemas con las guías
Desarrollo de Software										
Desarrollo código cámara	Vazquez	0%	3-9-24	27-9-24	60	-	05-09-24	13-12-24	60	Problema con el reconocimiento, distribución distinta de las horas
Desarrollo código cajones	Salto	0%	23-9-24	3-10-24	20	-	20-09-24	25-09-24	15	Ninguno
Desarrollo código dispenser	Salto	0%	3-9-24	23-9-24	20	-	07-09-24	15-09-24	10	Ninguno
Desarrollo código sensores de nivel	Salto	0%	23-9-24	30-9-24	10	-	27-09-24	27-09-24	3	Ninguno
Desarrollo código parlantes	Vazquez	0%	23-9-24	30-9-24	10	-	13-12-24	13-12-24	2	Se desarrolló luego de la cursada
Desarrollo de Aplicación										
Desarrollo función carga de imágenes y asignación de plato	Vazquez	0%	30-9-24	7-10-24	20	Desarrollo código cámara	-	-		No se realizó por la dificultad de coordinar la suba de fotos en la aplicación y el entrenamiento del modelo con esas fotos
Desarrollo función horario	Salto	0%	7-10-24	11-10-24	10	Desarrollo código dispenser	07-12-24	07-12-24	1	Ninguno
Desarrollo función ayuno	Salto	0%	11-10-24	14-10-24	10	Desarrollo código dispenser	07-12-24	07-12-24	1	Ninguno

Desarrollo función llamado	Vazquez	0%	14-10-24	21-10-24	10	Desarrollo código cámara, Desarrollo código parlantes	-	-		No se realizó por la dificultad de coordinar la suba del audio y su reproducción
Integración aplicación y software controlador	Salto y Vazquez	0%	21-10-24	5-11-24	30	Desarrollo de Software	07-12-24	08-12-24	8	Ninguno
Testeo										
Prueba mecanismo cajón	Vazquez	0%	3-10-24	10-10-24	8	Desarrollo de Software, Desarrollo mecánico	30-09-24	30-09-24	10	-
Prueba mecanismo cajón + cámara	Vazquez	0%	3-10-24	10-10-24	8	Desarrollo de Software, Desarrollo mecánico	30-09-24	13-12-24	15	-
Prueba mecanismo dispenser	Salto	0%	23-9-24	30-9-24	8	Desarrollo de Software, Desarrollo mecánico	05-11-24	07-11-24	15	-
Prueba mecanismo dispenser + celdas de carga	Salto	0%	23-9-24	30-9-24	8	Desarrollo de Software, Desarrollo mecánico	10-11-24	28-11-24	15	-
Prueba parlante + aplicación	Salto	0%	21-10-24	28-10-24	8	Desarrollo de Software, Desarrollo mecánico, Desarrollo de aplicación	13-12-24	13-12-24	2	-
Pruebas funciones aplicación	Salto y Vazquez	0%	7-10-24	14-11-24	50	Desarrollo de Software, Desarrollo mecánico, Desarrollo de aplicación	07-12-24	13-12-24	50	-
Pruebas finales	Salto y Vazquez	0%	14-11-24	21-11-24	40	Desarrollo de Software, Desarrollo mecánico, Desarrollo de aplicación	21-11-24	13-12-24	80	-
HORAS TOTALES			ESTIMADAS		437	REALES		465		

Puede verse que el proyecto tenía una fecha estimada de finalización que no fue cumplida, sino que el mismo se extendió dos semanas más de lo provisto. Además, muchas tareas que tenían fecha de finalización establecida no fueron terminadas hasta una vez concluido el proyecto, pues se necesitaba seguir corrigiendo cosas debido a los nuevos problemas que iban surgiendo. La tarea con mayor desvío en cuanto a fechas fue la de Construcción de la estructura, que debía terminar el 15-09-24 y no fue finalizada hasta el último día. En cuanto a horas, la tarea con mayor desviación fue la de Pruebas finales, que tenía una duración estimada de 40 horas, y su duración real terminó siendo el doble, ya que se al ir probando distintos escenarios se descubrían nuevos problemas, y la solución de los mismos fue computada como horas dentro de esta tarea. Sin embargo, también hay tareas que tenían una duración estimada que fue mayor a la real, como el desarrollo del código de los parlantes o de los sensores de nivel, que se estimaba duren 10 horas cada uno y se terminaron en 3 horas, o el desarrollo del código del dispenser, que se esperaba dure 20 horas y duró sólo la mitad.

FUNCIONAMIENTO

En primer lugar, para el funcionamiento del comedero, el usuario debe cargar en la aplicación las raciones y los horarios de la comida. La aplicación fue hecha utilizando AppSheets de Google, lo que simplificó el traslado de estos datos a los controladores. En las figuras 7 a 12 pueden verse pantallazos de la app creada.



Figura 7. Pantalla de espera de la app.

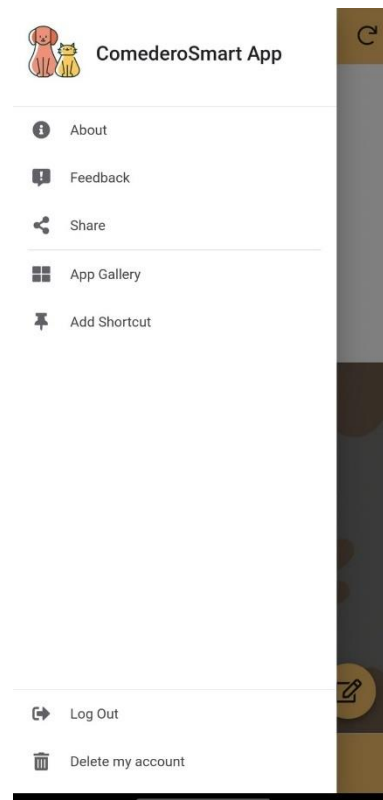


Figura 8. ComederoSmart App.

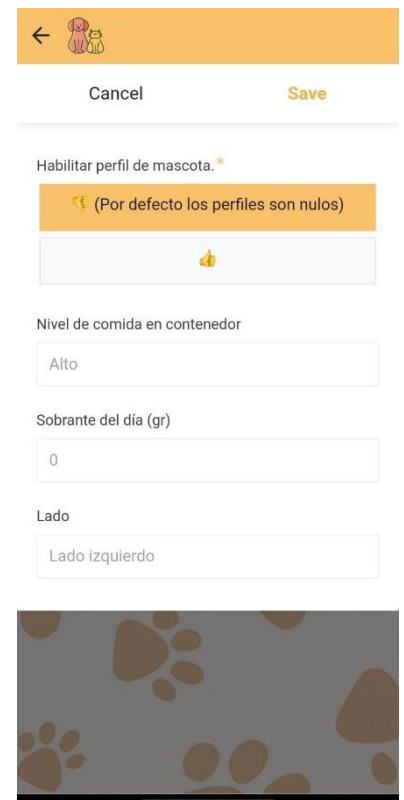


Figura 9. Menú para configurar el comedero.

Figura 10. Menú para agregar una nueva mascota.

Figura 11. Pantalla de control para la mascota del lado derecho.

Figura 12. Pantalla de control para la mascota del lado izquierdo.

Las funciones principales, como el movimiento de los cajones y de los tornillos sinfín están controladas por la ESP32, que recoge los valores de la app y realiza los ajustes necesarios. El código toma el horario actual gracias a una librería, y establece la función preparar comida unos minutos antes, con la porción indicada. Además, lleva registro del sobrante de comida en el plato. Por otro lado, los cajones sólo se abren si el plato tiene comida y la cámara reconoce a la mascota correcta frente a él, y se cierra si se acaba la porción o la mascota se va, o si se acerca la otra.

Para el reconocimiento de las mascotas, las dos placas ESP32-CAM envían fotos cada vez que detectan movimiento a una carpeta en Google Drive. Estas fotos son tomadas por el modelo de visión artificial realizado en Python que corre en Google Colab, y a medida que realiza la predicción mueve las fotos a otra carpeta dentro de Google Drive. Las predicciones hechas son escritas en la planilla de Sheets de la que se toman los datos en la ESP32.

Además, las cámaras tienen la función de medición de nivel del sensor ultrasónico, que si la distancia es menor a 5 cm reconoce el nivel de comida como Alto y lo plasma en la planilla de Sheets, mientras que si es mayor a 5 cm pero menor a 12 cm lo reconoce como medio y si es mayor a 12 cm lo reconoce como bajo. Si el nivel de comida es bajo, la función de dispensar comida no se ejecutará.

TESTEO

Las pruebas realizadas sobre el prototipo fueron las siguientes:

Mecanismo del dispenser: se hizo girar el tornillo sin fin para comprobar que la comida no lo trabara. Se observó que la comida no lo trababa, pero sí lo sacaba de su eje, lo que provocaba que se pierda el contacto con el piñón, y por eso dejaba de girar. Se solucionó fijando el tornillo sin fin a la pared de la estructura con un rodamiento, que le permitía rotar pero no moverse axialmente. Además, se probaron los sensores de ultrasonido, para ver si medían correctamente el nivel de alimento en el contenedor. Se encontró que cumplían correctamente su función.

Calibración de celdas de carga: se realizaron pruebas para verificar que las mediciones de la celda de carga fueran correctas al nivel que se necesitaba. Además, se verificó que la comida dispensada por el tornillo sin fin sea la cantidad correcta. Ambas pruebas se realizaron con una balanza auxiliar, y se comprobó que medían correctamente.

Movimiento de los cajones: se testeó que los topes de los cajones activen correctamente los finales de carrera. Se corrió un código que simplemente movía cada cajón hasta el final y luego esperaba 20 segundos para volver. Se tuvieron que modificar las guías utilizadas porque el juego que tenían con el cajón hacía que el mismo se desvíe y no logre hacer contacto con los sensores. Además, se testeó que, si el cajón está cerrado o abierto y se intenta sacarlo de su posición, el mismo vuelve a su lugar.

Reconocimiento de los animales: se comprobó que el programa sólo reconozca a la mascota indicada en cada caso. Se vio que con las dos mascotas utilizadas no se confundía, pero sí con imágenes de internet. Se decidió entrenar el modelo con una mayor cantidad de imágenes para aumentar su precisión. Además, se encontró que, como en un principio la cámara enviaba una foto cada 5 segundos a la carpeta de Drive, se saturaba el modelo que las analizaba. Esto se solucionó agregando un filtro previo en el código de las ESP32 CAM, en el que se analizaba si la imagen actual cambió algo con respecto a la anterior. Si se detectaron cambios, entonces la envía, y si no se detectaron cambios entonces la descarta y toma otra foto. Por último, se notó que el cajón se cerraba cuando el animal se agachaba para comer, ya que la cámara dejaba de verlo. La solución utilizada fue entrenar el modelo también con fotos de los animales en posición de comer, y bajar las cámaras para que los capten en ese ángulo.

Aplicación: se testeó que los cambios realizados en la App se plasmen en la base de datos de la que toma los valores el programa. Se comprobó que esto era así, verificando que la aplicación móvil funciona correctamente.

CONCLUSIONES

El prototipo construido logró las funcionalidades propuestas, sobreponiéndose a los problemas surgidos a lo largo del proyecto. El primer problema enfrentado fue el movimiento poco fluido y obstaculizado de los cajones, que se solucionó reemplazando las guías utilizadas en un principio por unas más grandes para disminuir el juego con el cajón. El segundo problema surgió a la hora de dispensar, dado que la comida ejercía demasiada fuerza sobre los tornillos sin fin, sacándolos de su eje y provocando que no se logren mover correctamente. Este problema fue solucionado colocando el eje en un rodamiento que fue fijado a la pared de la estructura, logrando que el tornillo sin fin sólo tenga libre el giro y restringiendo los demás grados de libertad.

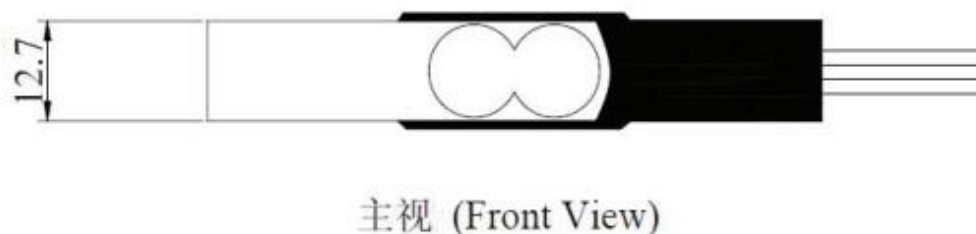
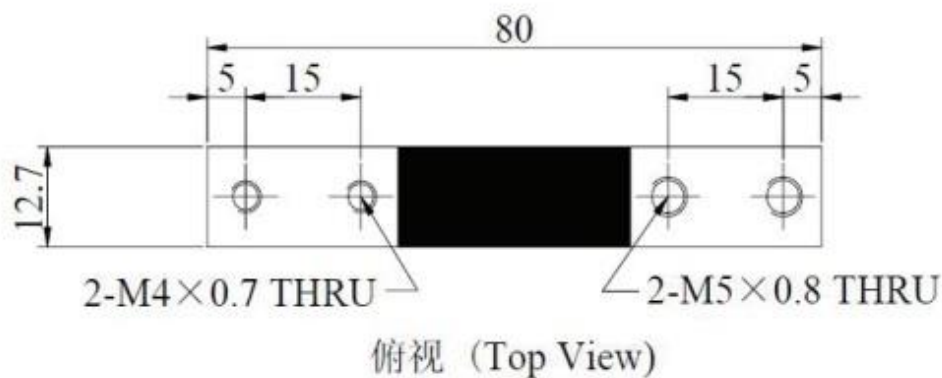
Por otro lado, se encontraron diversos aspectos a profesionalizar si se quisiera a lanzar el producto al mercado:

- En lugar de utilizar un modelo de visión artificial con las cámaras, sería más conveniente incluir con el comedero collares con alguna insignia NFC o Bluetooth o similar, que se enlace con el cajón y así se gana mucha precisión y se mejora el tiempo de procesado. En oposición a utilizar IA, este método tendría una confianza del 100%, ya que cada enlace sería único. Si se quisiera continuar con el modelo de visión artificial, sin embargo, se debería invertir en algún servidor de CloudComputing, para poder replicar el proceso realizado en Colab para varios comederos de forma más eficiente.
- En cuanto a estética se recomendaría cambiar en primer lugar el material, ya que la madera no es hipoalergénica y se están manipulando alimentos. Además, sería necesario hacerlo un poco más alto, para incrementar la capacidad de almacenamiento del mismo.
- Para la aplicación móvil se debería de crear un registro de usuarios, para garantizar seguridad y poder enlazar correctamente cada comedero con su respectivo usuario.

En resumen, podemos asegurar que el proyecto realizado nos permitió poner en práctica los conocimientos adquiridos a lo largo de nuestra formación académica, articulando tanto el área mecánica con el movimiento de los cajones y del dispenser, como también el área electrónica, utilizando diversos sensores y manejando todos los procesos desde el microcontrolador ESP32. Además, nos permitió incursionar en una nueva área de interés, como lo es la Inteligencia Artificial que está en auge en la actualidad. Fue un proyecto enriquecedor para nuestra formación, que nos ha permitido equivocarnos y nos ha obligado a encontrar cómo corregir esos errores, logrando que desarrollemos herramientas útiles para nuestro futuro profesional.

ANEXO 1: Hojas de datos de los componentes utilizados

CELDA DE CARGA



Especificación NA27 :

Capacidades nominales	0.6 , 1 , 2 , 3 , 5 , 6 , 10 (kg)
Salida nominal	1.0 ± 0.15mV / V
Saldo cero	± 0.1000 mV / V
No linealidad	0.03% R.O.
Histéresis	0.03% R.O.
Repetibilidad	0.03% R.O.
30 minutos de arrastramiento	0.03% R.O.
Retorno de 30mins	0.03% R.O.
Sobrecarga segura	150% de r.
Última sobrecarga	200% de r.
Efecto de la temperatura en la salida	0.01% R.O. / °C
Efecto de la temperatura en cero	0.05% R.O. / °C
Impedancia de entrada	1115 ± 10%
Impedancia de salida	1000 ± 10%
Impedancia de aislamiento	≥2000 MΩ / (50VDC)
Excitación Recomendada	5 ~ 12 VDC
Excitación máxima	15 VDC
Rango de temperatura de funcionamiento	-20 ~ 60 °C
Construcción	Aleación de aluminio
Clase de protección	IP65
Cable	φ0.8 × 0.25m
Tamaño de plataforma recomendado	150 × 150 mm
Modo de conexión	rojo (EXC +) , Negro (EXC -) , Verde (SIG +) , Blanco (SIG-)

Amplificador HX711:



HX711

KEY ELECTRICAL CHARACTERISTICS

Parameter	Notes	MIN	TYP	MAX	UNIT
Full scale differential input range	V(inp)-V(inn)	$\pm 0.5(AVDD/GAIN)$			V
Common mode input		AGND+1.2		AVDD-1.3	V
Output data rate	Internal Oscillator, RATE = 0		10		Hz
	Internal Oscillator, RATE = DVDD		80		
	Crystal or external clock, RATE = 0		$f_{clk}/1,105,920$		
	Crystal or external clock, RATE = DVDD		$f_{clk}/138,240$		
Output data coding	2's complement	800000		7FFFFF	HEX
Output settling time ⁽¹⁾	RATE = 0		400		ms
	RATE = DVDD		50		
Input offset drift	Gain = 128		0.2		mV
	Gain = 64		0.4		
Input noise	Gain = 128, RATE = 0		50		nV(rms)
	Gain = 128, RATE = DVDD		90		
Temperature drift	Input offset (Gain = 128)		± 6		nV/°C
	Gain (Gain = 128)		± 5		ppm/°C
Input common mode rejection	Gain = 128, RATE = 0		100		dB
Power supply rejection	Gain = 128, RATE = 0		100		dB
Reference bypass (V _{BG})			1.25		V
Crystal or external clock frequency		1	11.0592	20	MHz
Power supply voltage	DVDD	2.6		5.5	V
	AVDD, VSUP	2.6		5.5	
Analog supply current (including regulator)	Normal		1400		μA
	Power down		0.3		
Digital supply current	Normal		100		μA
	Power down		0.2		

(1) Settling time refers to the time from power up, reset, input channel change and gain change to valid stable output data.

Table 2 Key Electrical Characteristics

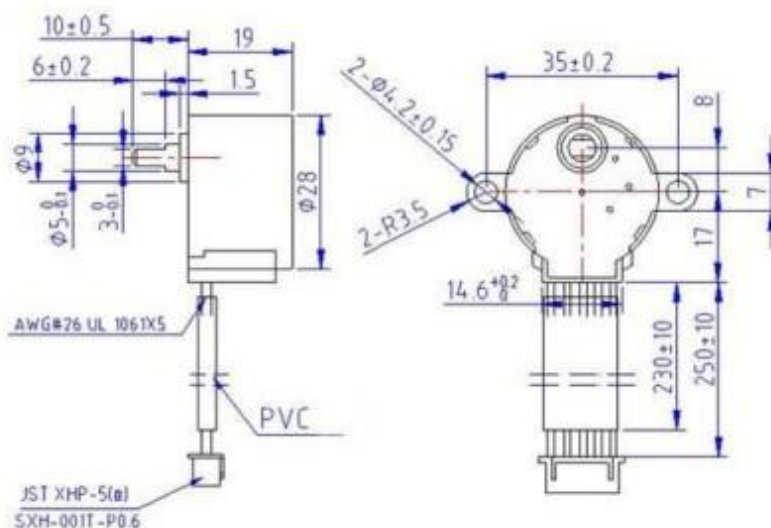
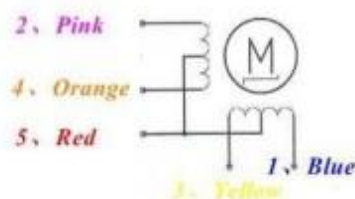
Motor paso a paso: 28BYJ-48

28BYJ-48 – 5V Stepper Motor

The 28BYJ-48 is a small stepper motor suitable for a large range of applications.



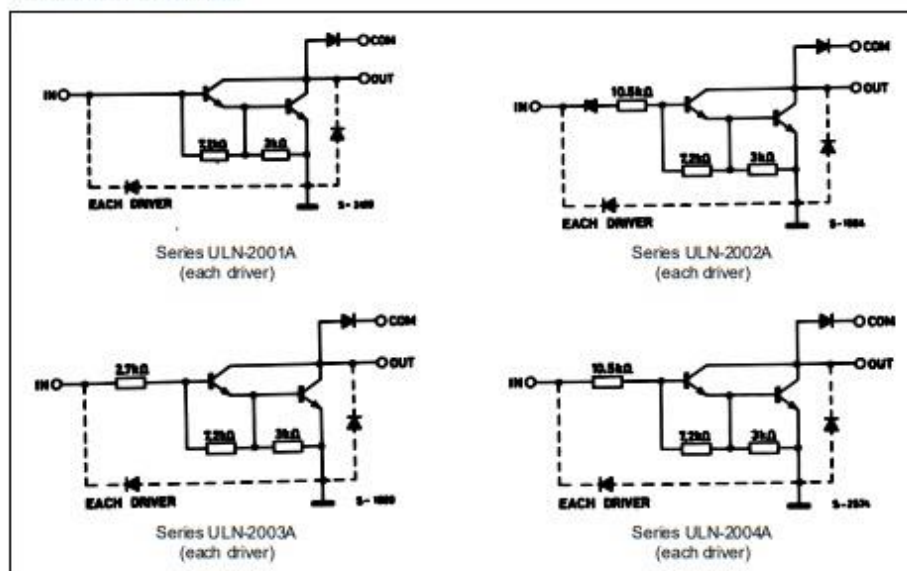
Rated voltage :	5VDC
Number of Phase	4
Speed Variation Ratio	1/64
Stride Angle	5.625°/64
Frequency	100Hz
DC resistance	50Ω±7%(25°C)
Idle In-traction Frequency	> 600Hz
Idle Out-traction Frequency	> 1000Hz
In-traction Torque	>34.3mN.m(120Hz)
Self-positioning Torque	>34.3mN.m
Friction torque	600-1200 gf.cm
Pull in torque	300 gf.cm
Insulated resistance	>10MΩ(500V)
Insulated electricity power	600VAC/1mA/1s
Insulation grade	A
Rise in Temperature	<40K(120Hz)
Noise	<35dB(120Hz, No load, 10cm)
Model	28BYJ-48 – 5V



Controlador motor paso a paso: ULN2003

ULN2001A - ULN2002A - ULN2003A - ULN2004A

SCHEMATIC DIAGRAM



ABSOLUTE MAXIMUM RATINGS

Symbol	Parameter	Value	Unit
V_o	Output Voltage	50	V
V_{in}	Input Voltage (for ULN2002A/D - 2003A/D - 2004A/D)	30	V
I_c	Continuous Collector Current	500	mA
I_b	Continuous Base Current	25	mA
T_{amb}	Operating Ambient Temperature Range	-20 to 85	°C
T_{stg}	Storage Temperature Range	-55 to 150	°C
T_j	Junction Temperature	150	°C

THERMAL DATA

Symbol	Parameter	DIP16	SO16	Unit
$R_{th \text{ j-amb}}$	Thermal Resistance Junction-ambient	Max. 70	120	°C/W

Sensor ultrasónico HC-SR04:

HC-SR04 Ultrasonic Sensor

Elijah J. Morgan

Nov. 16 2014

The purpose of this file is to explain how the HC-SR04 works. It will give a brief explanation of how ultrasonic sensors work in general. It will also explain how to wire the sensor up to a microcontroller and how to take/interpret readings. It will also discuss some sources of errors and bad readings.

1. How Ultrasonic Sensors Work
2. HC-SR04 Specifications
3. Timing chart, Pin explanations and Taking Distance Measurements
4. Wiring HC-SR04 with a microcontroller
5. Errors and Bad Readings



1. How Ultrasonic Sensors Work

Ultrasonic sensors use sound to determine the distance between the sensor and the closest object in its path. How do ultrasonic sensors do this? Ultrasonic sensors are essentially sound sensors, but they operate at a frequency above human hearing.

2. HC-SR04 Specifications

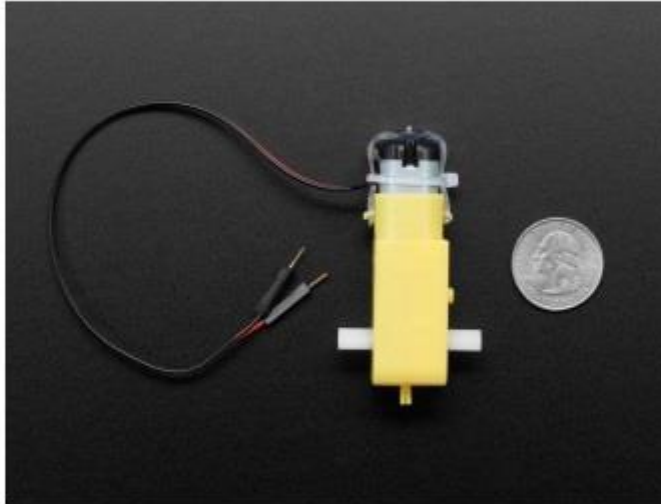
The sensor chosen for the Firefighting Drone Project was the HC-SR04. This section contains the specifications and why they are important to the sensor module. The sensor modules requirements are as follows.

- Cost
- Weight
- Community of hobbyists and support
- Accuracy of object detection
- Probability of working in a smoky environment
- Ease of use

The HC-SR04 Specifications are listed below. These specifications are from the Cytron Technologies HC-SR04 User's Manual (source 1).

- Power Supply: +5V DC
- Quiescent Current: <2mA
- Working current: 15mA
- Effectual Angle: <15°
- Ranging Distance: 2-400 cm
- Resolution: 0.3 cm
- Measuring Angle: 30°
- Trigger Input Pulse width: 10uS
- Dimension: 45mm x 20mm x 15mm
- Weight: approx. 10 g

Motor CC:



- At 3VDC we measured 150mA @ 120 RPM no-load, and 1.1 Amps when stalled
- At 4.5VDC we measured 155mA @ 185 RPM no-load, and 1.2 Amps when stalled
- At 6VDC we measured 160mA @ 250 RPM no-load, and 1.5 Amps when stalled

TECHNICAL DETAILS

- Rated Voltage: 3~6V
- Continuous No-Load Current: 150mA +/- 10%
- Min. Operating Speed (3V): 90 +/- 10% RPM
- Min. Operating Speed (6V): 200 +/- 10% RPM
- Torque: 0.15Nm ~0.60Nm
- Stall Torque (6V): 0.8kg.cm
- Gear Ratio: 1:48
- Body Dimensions: 70 x 22 x 18mm
- Wires Length: 200mm & 28 AWG
- Weight: 30.6g

Product Weight: 30.6g / 1.1oz

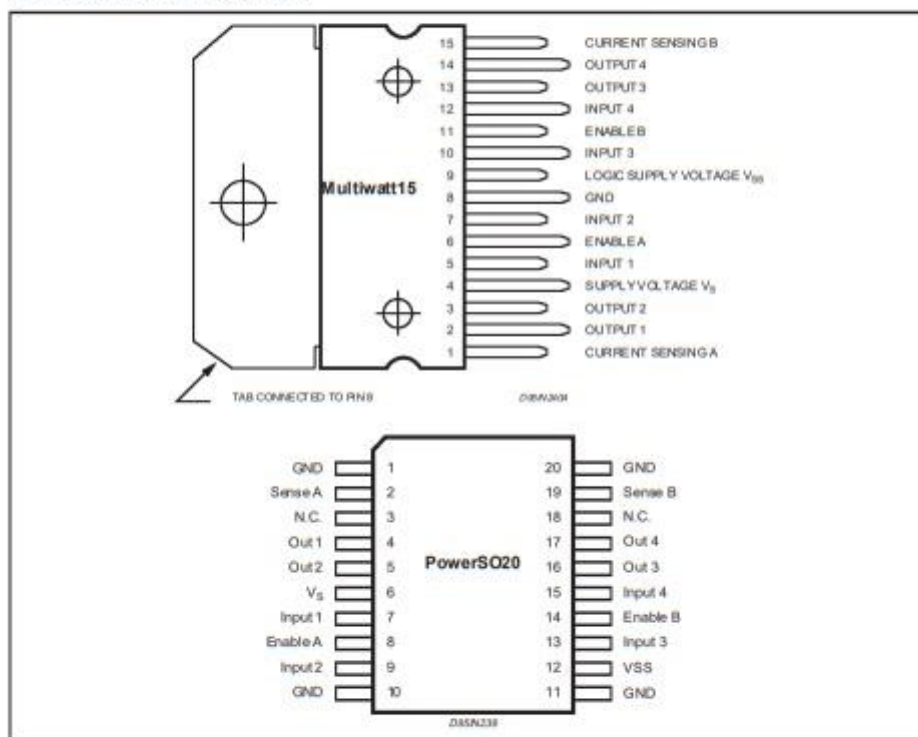
Controlador motor CC: L298N

L298

ABSOLUTE MAXIMUM RATINGS

Symbol	Parameter	Value	Unit
V_S	Power Supply	50	V
V_{SS}	Logic Supply Voltage	7	V
V_i, V_{en}	Input and Enable Voltage	-0.3 to 7	V
I_O	Peak Output Current (each Channel) - Non Repetitive ($t = 100\mu s$) - Repetitive (80% on -20% off; $t_{on} = 10ms$) - DC Operation	3 2.5 2	A A A
V_{sens}	Sensing Voltage	-1 to 2.3	V
P_{tot}	Total Power Dissipation ($T_{case} = 75^\circ C$)	25	W
T_{op}	Junction Operating Temperature	-25 to 130	$^\circ C$
T_{stg}, T_j	Storage and Junction Temperature	-40 to 150	$^\circ C$

PIN CONNECTIONS (top view)



ESP32-CAM:



EAN 4260474035731
February 17, 2021
Document version 1.0



Specifications:

SPI Flash default 32Mbit
RAM Internal 520kb + external 8MB PSRAM
Bluetooth Bluetooth 4.2 BR / EDR and BLE standard
Wi-Fi 802.11 b/g/n/e/i
Supported interfaces UART, SPI, I2C, PWM
Serial port rate default 115200 bps
Image output format JPEG (OV2640 support only), BMP, Grayscale
Antenna on board (PCB antenna), gain 2dbi
Transmit power:
802.11b: 17 ± 2 dBm (@11Mbps)
802.11g: 14 ± 2 dBm (@54Mbps)
802.11n: 13 ± 2 dBm (@MCS7)

Receive sensitivity:
CCK, 1 Mbps: -90dBm
CCK, 11 Mbps: -85dBm
6 Mbps (1/2 BPSK): -88dBm
54 Mbps (3/4 64-QAM): -70DBM
MCS7 (65 Mbps, 72.2 Mbps): -67dBm
Power consumption:
w/o Flash: 180mA @ 5V
with Flash (full brightness): 310mA @ 5V
Deep-Sleep: 6mA @ 5V
Security: WPA/WPA2/WPA2-Enterprise/WPS
Power supply range 4.75-5.25V
Operating temperature $-20^{\circ}\text{C} \sim 70^{\circ}\text{C}$
Storage environment $-40^{\circ}\text{C} \sim 125^{\circ}\text{C}$
Size $27 \times 40.5 \times 4.5$ (± 0.2) mm

Lente ESP32-CAM: OV2640



Advanced Information
Preliminary Datasheet

OV2640 Color CMOS UXGA (2.0 MegaPixel) CAMERACHIP™ with OmniPixel2™ Technology

General Description

The OV2640 CAMERACHIP™ is a low voltage CMOS image sensor that provides the full functionality of a single-chip UXGA (1632x1232) camera and image processor in a small footprint package. The OV2640 provides full-frame, sub-sampled, scaled or windowed 8-bit/10-bit images in a wide range of formats, controlled through the Serial Camera Control Bus (SCCB) interface.

This product has an image array capable of operating at up to 15 frames per second (fps) in UXGA resolution with complete user control over image quality, formatting and output data transfer. All required image processing functions, including exposure control, gamma, white balance, color saturation, hue control, white pixel canceling, noise canceling, and more, are also programmable through the SCCB interface. The OV2640 also includes a compression engine for increased processing power. In addition, OmniVision CAMERACHIPs use proprietary sensor technology to improve image quality by reducing or eliminating common lighting/electrical sources of image contamination, such as fixed pattern noise, smearing, etc., to produce a clean, fully stable color image.



Note: The OV2640 uses a lead-free package.

Features

- High sensitivity for low-light operation
- Low operating voltage for embedded portable apps
- Standard SCCB interface
- Output support for Raw RGB, RGB (RGB565/555), GRB422, YUV (422/420) and YCbCr (4:2:2) formats
- Supports image sizes: UXGA, SXGA, SVGA, and any size scaling down from SXGA to 40x30
- VarioPixel® method for sub-sampling
- Automatic image control functions including Automatic Exposure Control (AEC), Automatic Gain Control (AGC), Automatic White Balance (AWB), Automatic Band Filter (ABF), and Automatic Black-Level Calibration (ABLC)
- Image quality controls including color saturation, gamma, sharpness (edge enhancement), lens correction, white pixel canceling, noise canceling, and 50/60 Hz luminance detection
- Line optical black level output capability
- Video or snapshot operation
- Zooming, panning, and windowing functions
- Internal/external frame synchronization
- Variable frame rate control
- Supports LED and flash strobe mode
- Supports scaling
- Supports compression
- Embedded microcontroller

Ordering Information

Product	Package
OV02640-VL9A (Color, lead-free)	38-pin CSP2

Applications

- Cellular and Camera Phones
- Toys
- PC Multimedia
- Digital Still Cameras

Key Specifications

Array Size	UXGA	1600 x 1200
Power Supply	Core	1.3VDC ± 5%
	Analog	2.5 ~ 3.0VDC
Power Requirements	I/O	1.7V to 3.3V
	Active	125 mW (for 15 fps, UXGA YUV mode)
	Standby	140 mW (for 15 fps, UXGA compressed mode)
Temperature Range	Stable Image	0°C to 50°C
Output Formats (8-bit)		• YUV(422/420)/YCbCr422
		• RGB565/555
		• 8-bit compressed data
		• 8-/10-bit Raw RGB data
Lens Size		
Chief Ray Angle		
Maximum Image Transfer Rate		
Sensitivity		
S/N Ratio		
Dynamic Range		
Scan Mode		
Maximum Exposure Interval		
Gamma Correction		
Pixel Size		
Dark Current		
Well Capacity		
Fixed Pattern Noise		
Image Area		
Package Dimensions		

Figure 1 OV2640 Pin Diagram (Top View)



ESP32:

Espressif Systems

Smart Documentation Feedback

14

ESP32 Series Datasheet v4.7

2.2 Pin Overview

Table 2-1. Pin Overview

Name	No.	Type	Function
Analog			
VDDA	1	P	Analog power supply (2.3 V ~ 3.6 V)
LNA_IN	2	I/O	RF input and output
VDD3P3	3	P	Analog power supply (2.3 V ~ 3.6 V)
VDD3P3	4	P	Analog power supply (2.3 V ~ 3.6 V)
VDD3P3_RTC			
SENSOR_VP	5	I	GPIO36, ADC1_CH0, RTC_GPIO0
SENSOR_CAPP	6	I	GPIO37, ADC1_CH1, RTC_GPIO1
SENSOR_CAPN	7	I	GPIO38, ADC1_CH2, RTC_GPIO2
SENSOR_VN	8	I	GPIO39, ADC1_CH3, RTC_GPIO3
CHIP_PU	9	I	High: On; enables the chip Low: Off; the chip shuts down Note: Do not leave the CHIP_PU pin floating.
VDET_1	10	I	GPIO34, ADC1_CH6, RTC_GPIO4
VDET_2	11	I	GPIO35, ADC1_CH7, RTC_GPIO5
32K_XP	12	I/O	GPIO32, ADC1_CH4, RTC_GPIO9, TOUCH9, 32K_XP (32.768 kHz crystal oscillator input)
32K_XN	13	I/O	GPIO33, ADC1_CH5, RTC_GPIO8, TOUCH8, 32K_XN (32.768 kHz crystal oscillator output)
GPIO25	14	I/O	GPIO25, ADC2_CH8, RTC_GPIO6, DAC_1, EMAC_RXD0
GPIO26	15	I/O	GPIO26, ADC2_CH9, RTC_GPIO7, DAC_2, EMAC_RXD1
GPIO27	16	I/O	GPIO27, ADC2_CH7, RTC_GPIO17, TOUCH7, EMAC_RX_DV
MTMS	17	I/O	GPIO14, ADC2_CH6, RTC_GPIO16, TOUCH6, EMAC_TXD2, HSPICLK, HS2_CLK, SD_CLK, MTMS
MTDI	18	I/O	GPIO12, ADC2_CH5, RTC_GPIO15, TOUCH5, EMAC_TXD3, HSPICLK, HS2_DATA2, SD_DATA2, MTDI
VDD3P3_RTC	19	P	Input power supply for RTC IO (2.3 V ~ 3.6 V)
MTCK	20	I/O	GPIO13, ADC2_CH4, RTC_GPIO14, TOUCH4, EMAC_RX_ER, HSPID, HS2_DATA3, SD_DATA3, MTCK
MTDO	21	I/O	GPIO15, ADC2_CH3, RTC_GPIO13, TOUCH3, EMAC_RXD3, HSPICLK, HS2_CMD, SD_CMD, MTDO

2 Pins

Espressif Systems

Smart Documentation Feedback

15

ESP32 Series Datasheet v4.7

Name	No.	Type	Function
GPIO2	22	I/O	GPIO2, ADC2_CH2, RTC_GPIO12, TOUCH2, HSPWP, HS2_DATA0, SD_DATA0
GPIO0	23	I/O	GPIO0, ADC2_CH1, RTC_GPIO11, TOUCH1, EMAC_TX_CLK, CLK_OUT1
GPIO4	24	I/O	GPIO4, ADC2_CH0, RTC_GPIO10, TOUCH0, EMAC_TX_ER, HSPID, HS2_DATA1, SD_DATA1
VDD_SDIO			
GPIO16	25	I/O	GPIO16, HS1_DATA4, U2RXD, EMAC_CLK_OUT
VDD_SDIO	26	P	Output power supply: 1.8 V or the same voltage as VDD3P3_RTC
GPIO17	27	I/O	GPIO17, HS1_DATA5, U2TXD, EMAC_CLK_OUT_180
SD_DATA_2	28	I/O	GPIO9, HS1_DATA2, U1RXD, SD_DATA2, SPIHD
SD_DATA_3	29	I/O	GPIO10, HS1_DATA3, U1TXD, SD_DATA3, SPIWP
SD_CMD	30	I/O	GPIO11, HS1_CMD, U1RTS, SD_CMD, SPICLK
SD_CLK	31	I/O	GPIO6, HS1_CLK, U1CTS, SD_CLK, SPICLK
SD_DATA_0	32	I/O	GPIO7, HS1_DATA0, U2RTS, SD_DATA0, SPIQ
SD_DATA_1	33	I/O	GPIO8, HS1_DATA1, U2CTS, SD_DATA1, SPID
VDD3P3_CPU			
GPIO5	34	I/O	GPIO5, HS1_DATA6, VSPICLK, EMAC_RX_CLK
GPIO18	35	I/O	GPIO18, HS1_DATA7, VSPICLK
GPIO23	36	I/O	GPIO23, HS1_STROBE, VSPID
VDD3P3_CPU	37	P	Input power supply for CPU IO (1.8 V ~ 3.6 V)
GPIO19	38	I/O	GPIO19, U0CTS, VSPID, EMAC_TXD0
GPIO22	39	I/O	GPIO22, U0RTS, VSPWP, EMAC_TXD1
U0RXD	40	I/O	GPIO3, U0RXD, CLK_OUT2
U0TXD	41	I/O	GPIO1, U0TXD, CLK_OUT3, EMAC_RXD2
GPIO21	42	I/O	GPIO21, VSPID, EMAC_TX_EN
Analog			
VDDA	43	P	Analog power supply (2.3 V ~ 3.6 V)
XTAL_N	44	O	External crystal output
XTAL_P	45	I	External crystal input
VDDA	46	P	Analog power supply (2.3 V ~ 3.6 V)
CAP2	47	I	Connects to a 3.3 nF (10%) capacitor and 20 kΩ resistor in parallel to CAP1

2 Pins

Name	No.	Type	Function
CAP1	48	I	Connects to a 10 nF series capacitor to ground
GND	49	P	Ground

2.3 IO Pins

2.3.1 Restrictions for GPIOs and RTC_GPIOs

All IO pins of the ESP32 have GPIO and some have RTC_GPIO pin functions. However, these IO pins are multifunctional and can be configured for different purposes based on the requirements. Some IOs have restrictions for usage. It is essential to consider their multiplexed nature and the limitations when using these IO pins.

In Table 2-1 Pin Overview some pin functions are highlighted, specially:

- **GPIO** - **Input only pins**, output is not supported due to lack of pull-up/pull-down resistors.
- **GPIO** - allocated for communication with in-package flash/PSRAM and NOT recommended for other uses. For details, see Section 2.6 Pin Mapping Between Chip and Flash/PSRAM.
- **GPIO** - have one of the following important functions:
 - **Strapping pins** - need to be at certain logic levels at startup. See Section 3 *Boot Configurations*.
 - **JTAG interface** - often used for debugging.
 - **UART interface** - often used for debugging.

See also [Appendix A.1 - Notes on ESP32 Pin Lists](#).